

Agendamento de Salão de Beleza

Uma arquitetura orientada a eventos

Confirmação de agendamento · ocupação de profissionais · agregação por janela · resiliência com Kafka

Equipe 07

O que o sistema resolve

- **Salão de beleza: confirmação, cobrança de sinal e ocupação dos profissionais**
- **Tudo orientado a eventos**
- **servico-agendamentos confirma → publica evento no tópico `salao.agendamento-confirmado`**
- **servico-ocupacao consome o mesmo evento para responder duas perguntas:**
 - “Qual a ocupação de cada profissional?”
 - “Quantos agendamentos por prioridade, em janelas de 15 min?”

salao.agendamento.confirmado.v1

- 7 campos, todos documentados: tipo, obrigatoriedade e significado
- Datas em ISO-8601 — nunca epoch
- Chave de partição: agendamentoid
- Compatibilidade: BACKWARD

```
"eventoId": "EVT-20260816-0001"  
"agendamentoId": "AG-20260822-0142"  
"profissionalId": "PROF-102"  
"servicoId": "SERV-004"  
"inicioEm": "2026-08-22T14:00:00-03:00"  
"prioridade": "URGENTE"  
"ocorridoEm":  
"2026-08-16T12:30:00-03:00"
```

Do pedido ao evento

- Cliente solicita um horário para um profissional e serviço.
- O serviço de agendamentos verifica disponibilidade e aplica a regra de prioridade.
- Após a confirmação, o agendamento passa a ocupar aquele horário.
- O fato é publicado como `salao.agendamento.confirmado.v1`.
- Outros serviços reagem ao evento sem precisar consultar o produtor.

Um evento, dois consumidores

Consumidor	O que faz	Relógio
Projeção de ocupação	Ocupação por profissional, por agendamento	—

Agregador por janela

Confirmações por prioridade, janelas de 15 min

event time

- Mesmo tópico, group.id diferentes → fan-out do Kafka: cada grupo recebe todas as mensagens
- Cada consumidor tem sua própria factory, com retry, backoff e DLQ configurados individualmente

Por que `agendamentoId` é a chave de partição?

- Eventos do mesmo agendamento ficam na mesma partition, preservando sua ordem.
- Agendamentos diferentes podem ser processados em paralelo.
- `eventId` não substitui `agendamentoId`: um identifica a ocorrência; o outro identifica o agendamento.
- Essa separação também ajuda na idempotência e no processamento distribuído.

O que significa **BACKWARD**?

- Um consumidor antigo continua funcionando quando o evento recebe um novo campo compatível.
- A evolução deve ser aditiva: adicionar informação nova sem mudar o significado do que já existe.
- Consumidores devem ignorar campos desconhecidos e usar apenas os campos necessários.
- Remover, renomear ou mudar o significado de um campo exige nova versão, como v2.

Reprocessamento manual, não automático

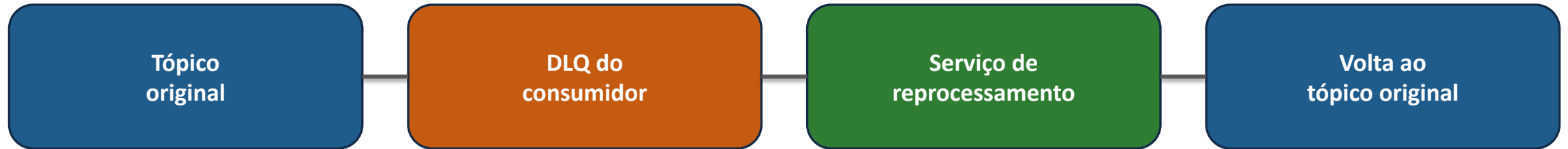
- Alternativa considerada: job automático reprocessando a DLQ sozinho
- Risco: loop de falha infinito se a causa raiz ainda não foi corrigida
- Decisão: reprocessamento sob controle humano — POST /admin/dlq/{consumidor}/reprocessar
- Consequência aceita: o sistema não se autocura sozinho

Retry com backoff exponencial



- Até 3 tentativas, backoff exponencial: 1s, 2s, 4s
- Serve para falha transiente (ex.: banco fora do ar por um instante)
- JSON malformatado é erro determinístico → direto para a DLQ, sem gastar retry

Dead Letter Queue e reproprocessamento



- Uma DLQ por consumidor, não compartilhada
- Offset só é confirmado depois da republicação confirmada — nunca antes
- `POST /admin/dlq/{consumidor}/reprocessar → {"reprocessados": N}`